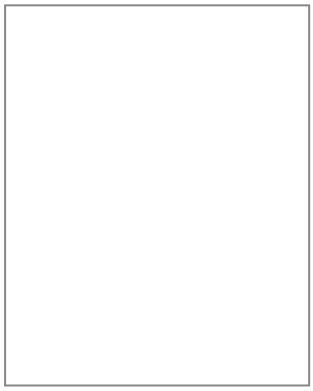
Escoge tu propia aventura

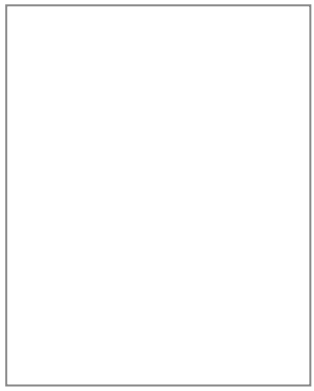
Obligatorio de Programación 3
Facultad de Ingeniería, Universidad ORT Uruguay

Grupo: _____



Nombre: _____

Nº de estudiante: _____



Nombre: _____

Nº de estudiante: _____

Noviembre de 2026

Tabla de contenido

1. Presentación del sistema **3**

1.1. Enlaces 3

1.2. Numeración de los requerimientos 3

2. Diagrama de casos de uso **4**

3. Descripción narrativa de casos de uso **5**

3.1. Lectura de historia (RF07) 5

3.2. Historial de lecturas (RF08) 6

4. Casos de prueba: ranking de historias más leídas (RF09) **7**

4.1. Clases de equivalencia 7

4.2. Casos de prueba 7

5. Diagramas de clases **8**

5.1. Dominio: entidades 9

5.2. Dominio: repositorios y consultas 10

5.3. Dominio: excepciones 11

5.4. Aplicación: servicios 12

5.5. Aplicación: DTOs 13

5.6. Aplicación: mappers 14

5.7. Lectura de historia: MVC y Web API 15

5.8. Lectura de historia: aplicación 16

5.9. Lectura de historia: repositorios e infraestructura 17

6. Generación de datos de prueba con IA generativa **18**

6.1. Captura de la conversación 18

6.2. Prompts utilizados 18

6.3. Del resultado de la IA al script 19

6.4. Controles 19

6.5. Filas por tabla 20

6.6. Datos para probar las métricas 20

7. Investigación para la selección de estilos (RF12) **22**

7.1. Herramienta y método 22

7.2. Qué había que resolver 22

7.3. Cómo cambiar los colores 22

7.4. Dónde guardar la elección 22

7.5. Contraste 23

7.6. Implementación 23

7.7. Decisiones y limitaciones 24

7.8. Fuentes 24

1. Presentación del sistema

El sistema permite publicar y leer historias interactivas, en las que cada lector elige cómo sigue la trama hasta llegar a uno de los finales posibles. Está formado por dos soluciones independientes que comparten las bibliotecas de clases:

Solución	Proyectos	Función
`Administracion.slnx`	`Obligatorio`, `Dominio`, `Aplicacion`, `Infraestructura`	Aplicación MVC de administración: usuarios, historias, capítulos y auditoría.
`Lectura.slnx`	`Obligatorio.WebApi`, `Obligatorio.Lector`, `Dominio`, `Aplicacion`, `Infraestructura`	Web API con JWT y su cliente MVC: lectura, historial, métricas y estilo.

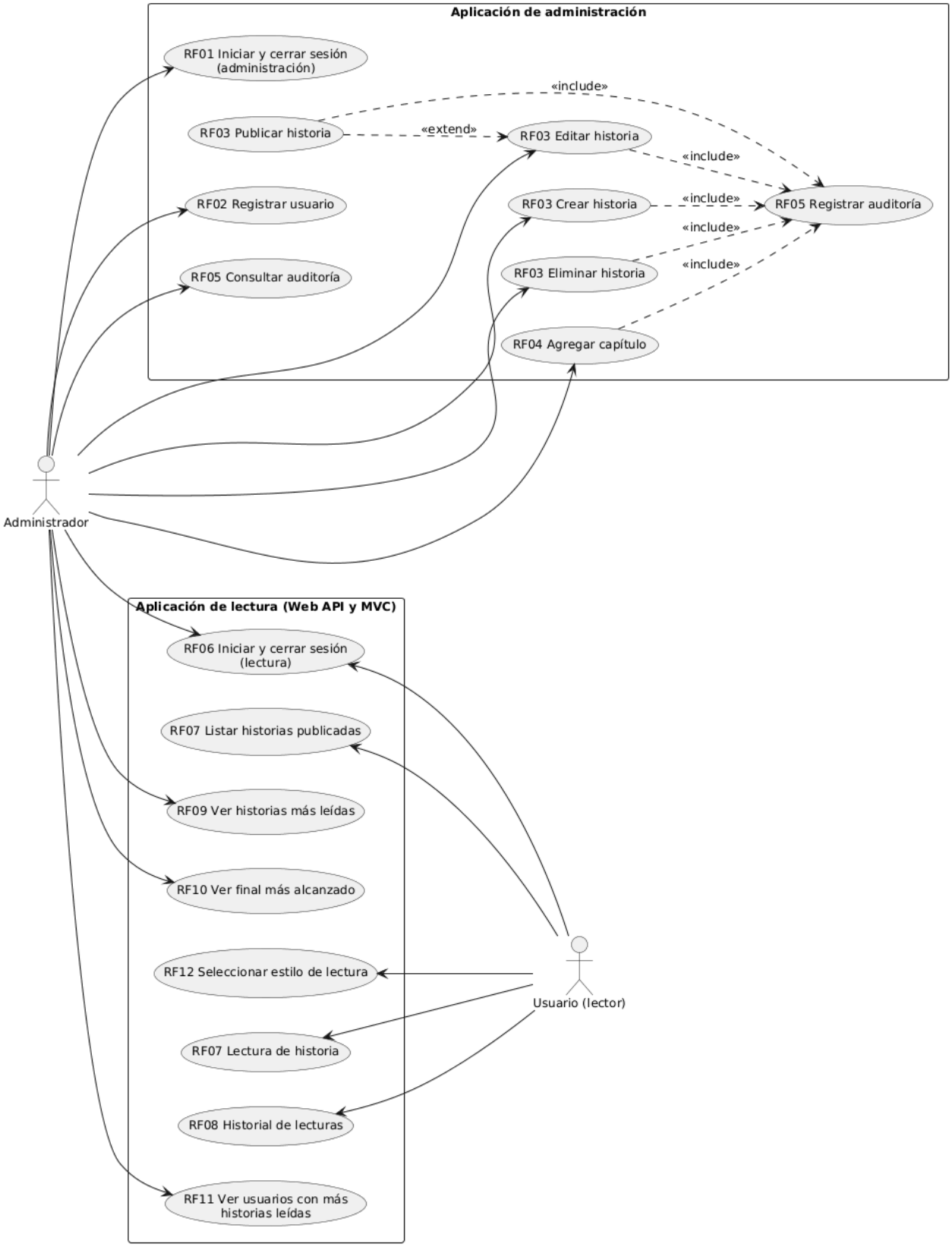
1.1. Enlaces

Administración (Somee)	http://escogetuaventura.somee.com/admin/
Lectura (Somee)	http://escogetuaventura.somee.com/
Web API y Swagger (Somee)	http://escogetuaventura.somee.com/webapi/swagger
Repositorio en GitHub	[completar: repositorio de GitHub]

1.2. Numeración de los requerimientos

La sección de documentación de la consigna cita algunos requerimientos con otro número (por ejemplo, RF06 para Lectura de historia). En este informe se usa la numeración de la descripción de los requerimientos: RF07 Lectura, RF08 Historial, RF09 Ranking de historias y RF12 Estilos.

2. Diagrama de casos de uso



Casos de uso de las dos aplicaciones del sistema.

3. Descripción narrativa de casos de uso

3.1. Lectura de historia (RF07)

Actor principal	Usuario con rol lector.
Objetivo	Recorrer una historia publicada eligiendo opciones hasta llegar a un final, que queda registrado.
Precondiciones	El usuario inició sesión y existe al menos una historia publicada.
Poscondiciones	Queda registrada una lectura con su capítulo actual. Si se llegó a un final, también quedan la fecha de finalización y el final alcanzado.

Flujo principal

Paso	Usuario	Sistema
1	Ingresa a Historias.	Muestra las historias publicadas y las categorías para filtrar.
2	Opcionalmente, elige una categoría y filtra.	Muestra solo las historias publicadas de esa categoría.
3	Elige Comenzar lectura en una historia.	Crea una lectura nueva, ubicada en el capítulo inicial, con la fecha actual.
4		Muestra el título y el texto del capítulo, y sus opciones.
5	Elige una opción.	Comprueba que pertenezca al capítulo actual y avanza al capítulo de destino.
6		Si el destino es intermedio, vuelve al paso 4.
7		Si el destino es un final, registra la fecha de finalización y el final alcanzado, y muestra el capítulo final con la opción de iniciar una nueva lectura.

Flujos alternativos

Paso	Condición	Respuesta del sistema
1a	No hay historias para el filtro elegido.	Informa: «No hay historias publicadas para mostrar con ese filtro.»
3a	La historia no existe.	Informa: «La historia no existe.» La API responde 404.
3b	La historia dejó de estar publicada.	Informa: «Solo se pueden leer historias publicadas.» La API responde 400.
4a	La lectura no existe o es de otro usuario.	Informa: «La lectura no existe o no pertenece al usuario.» La API responde 404.
5a	La opción no pertenece al capítulo actual.	Informa: «La opción seleccionada no pertenece al capítulo actual.» La API responde 400.
5b	La lectura ya había finalizado.	Informa: «La lectura ya finalizó. Debe iniciar una nueva.» La API responde 400.
7a	El usuario quiere probar otro camino.	Elige Iniciar una nueva lectura y vuelve al paso 3. La lectura anterior no cambia.

Paso	Condición	Respuesta del sistema
*a	La sesión venció o el token no es válido.	La API responde 401 y la aplicación vuelve al inicio de sesión.
*b	Un administrador intenta leer.	La API responde 403 y la aplicación informa que no tiene permisos.
*c	La API no está disponible.	Informa: «No se pudo conectar con la API.»

3.2. Historial de lecturas (RF08)

Actor principal	Usuario con rol lector.
Objetivo	Consultar las lecturas propias iniciadas en un mes y año, con su estado y el final alcanzado.
Precondiciones	El usuario inició sesión.
Poscondiciones	No se modifica ningún dato.

Flujo principal

Paso	Usuario	Sistema
1	Ingresa a Mi historial.	Selecciona el mes y el año actuales, y muestra las lecturas de ese período.
2	Elige otro mes y año, y busca.	Busca solo las lecturas de ese usuario iniciadas en ese período.
3		Muestra, de la más reciente a la más antigua: historia, fecha de inicio, estado, fecha de finalización y final alcanzado.
4	Opcionalmente, elige Continuar en una lectura en curso.	Muestra el capítulo donde había quedado (continúa en el caso de uso Lectura de historia, paso 4).

Flujos alternativos

Paso	Condición	Respuesta del sistema
2a	No hay lecturas en el período.	Informa: «No existen lecturas para el mes y año seleccionados.» La API responde 200 con una lista vacía.
2b	El mes no está entre 1 y 12.	Informa: «El mes debe estar entre 1 y 12.» La API responde 400.
2c	El año no es válido.	Informa: «El año ingresado no es válido.» La API responde 400.
*a	La sesión venció o el token no es válido.	La API responde 401 y la aplicación vuelve al inicio de sesión.
*b	Un administrador intenta consultar.	La API responde 403 y la aplicación informa que no tiene permisos.

4. Casos de prueba: ranking de historias más leídas (RF09)

Requerimiento: listado de historias ordenado por cantidad de lecturas completadas, de mayor a menor, sin las historias nunca leídas. Solo para administradores. Se usa diseño de caja negra, con clases de equivalencia sobre el estado de las lecturas y valores límite sobre la cantidad.

Se prueba con `GET /api/Metricas/ranking-historias` y con la pantalla Historias más leídas de la aplicación de lectura. Estos casos se ejecutaron de forma automatizada contra la aplicación en funcionamiento y dieron el resultado esperado.

4.1. Clases de equivalencia

Condición	Clases válidas: la historia aparece	Clases inválidas: la historia no aparece
Lecturas de la historia	C1: al menos una completada.	C2: ninguna lectura. C3: solo lecturas en curso.
Rol del solicitante	C4: administrador.	C5: lector (403). C6: sin autenticar (401).

4.2. Casos de prueba

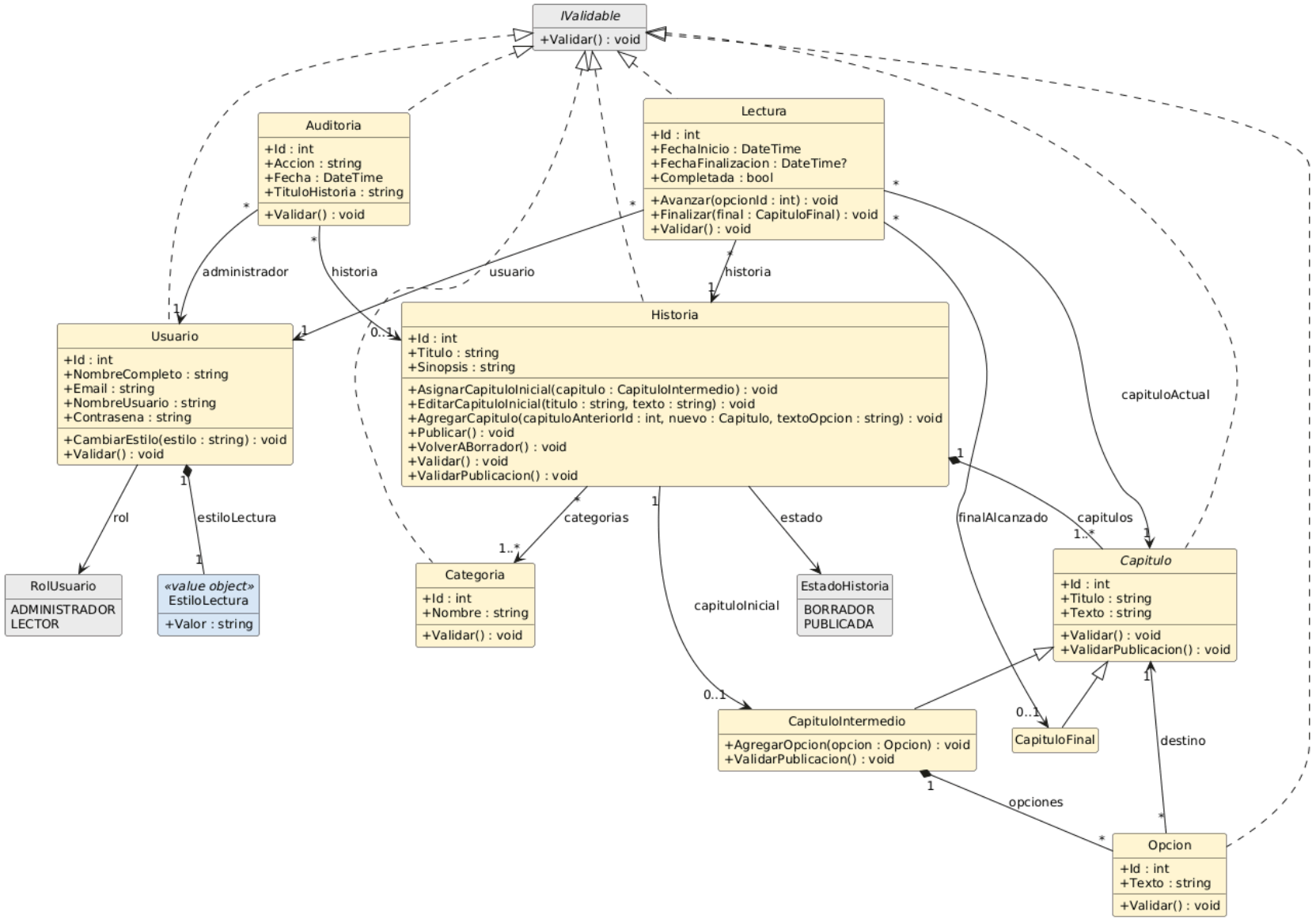
Id	Caso	Datos de entrada	Resultado esperado
CP01	Sin lecturas completadas	Base con historias publicadas y ninguna lectura.	API: 200 y lista vacía. Pantalla: «Todavía no hay lecturas completadas.»
CP02	Historia nunca leída (C2)	Delta sin lecturas y Alfa con 1 completada.	Aparece solo Alfa.
CP03	Solo lecturas en curso (C3)	Gama con 1 lectura sin terminar y Alfa con 1 completada.	Gama no aparece.
CP04	Mínimo: una completada (C1)	Alfa con exactamente 1 completada.	Alfa aparece con cantidad 1.
CP05	Orden descendente	Alfa con 4 y Beta con 2.	Alfa, Beta.
CP06	Empate	Beta con 2 y Epsilon con 2.	Ambas con 2, ordenadas alfabéticamente: Beta, Epsilon.
CP07	Completadas y en curso juntas	Alfa con 4 completadas y 1 en curso.	Alfa aparece con cantidad 4.
CP08	El mismo usuario repite	Un usuario completa Alfa 2 veces.	Cuentan las 2 lecturas: se cuentan lecturas, no usuarios.
CP09	Se actualiza con el uso	Ranking vacío; un lector completa una historia.	La historia pasa a aparecer con cantidad 1.
CP10	Solicitante lector (C5)	GET con token de lector.	403. La pantalla informa que no tiene permisos.
CP11	Sin autenticar (C6)	GET sin token.	401. La pantalla lleva al inicio de sesión.

5. Diagramas de clases

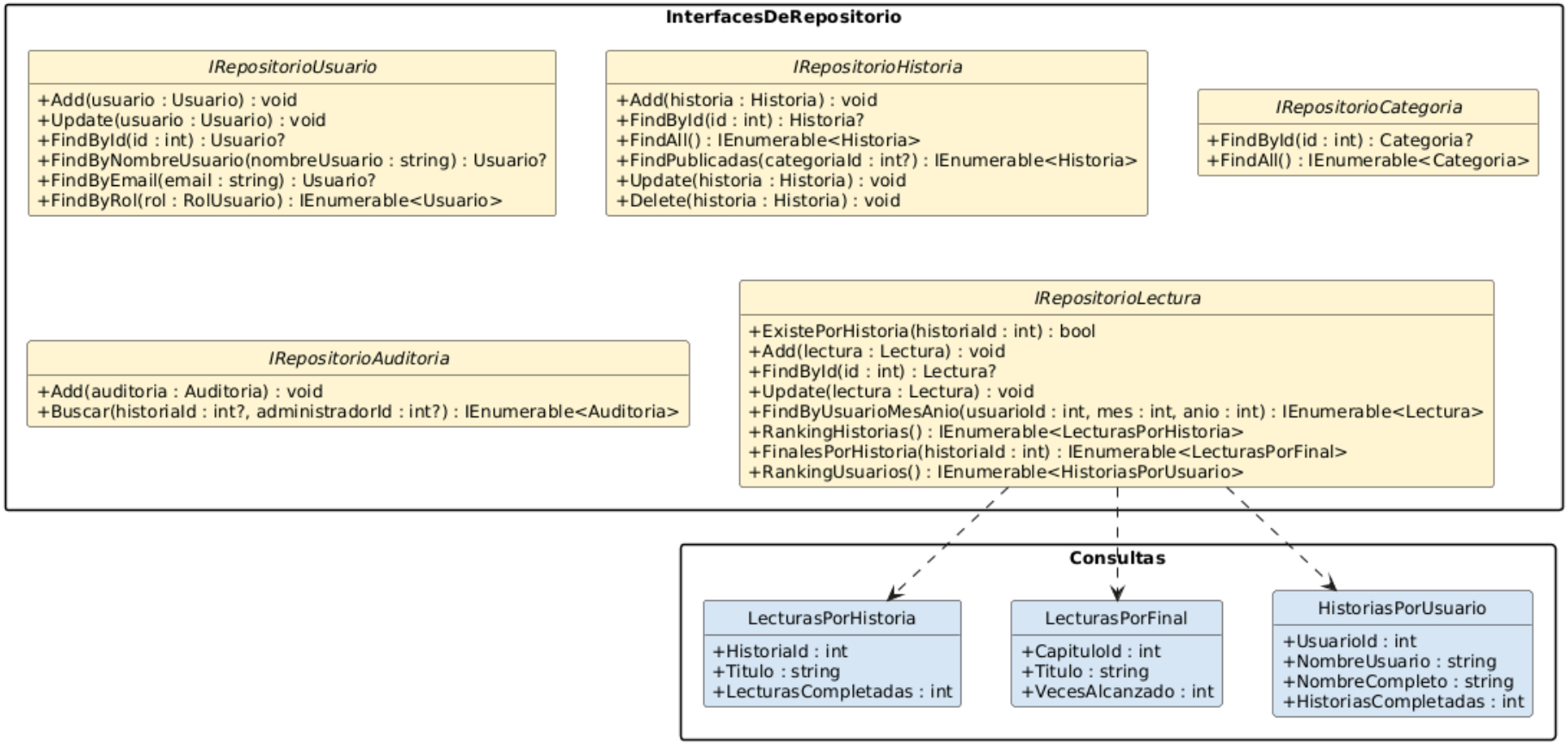
Los diagramas se hicieron con PlantUML online y se incluyen como capturas de pantalla. Los atributos y operaciones se generan leyendo el código, por lo que coinciden con lo implementado. Se incluye completa la lógica de negocio (Dominio y Aplicación) y, para las demás capas, las clases que participan en Lectura de historia, sin las vistas MVC.

Respecto al diseño de las pre-entregas, las veces que se alcanzó un final ya no se guardan en `CapituloFinal`: se cuentan desde las lecturas, para no mantener el mismo dato en dos lugares.

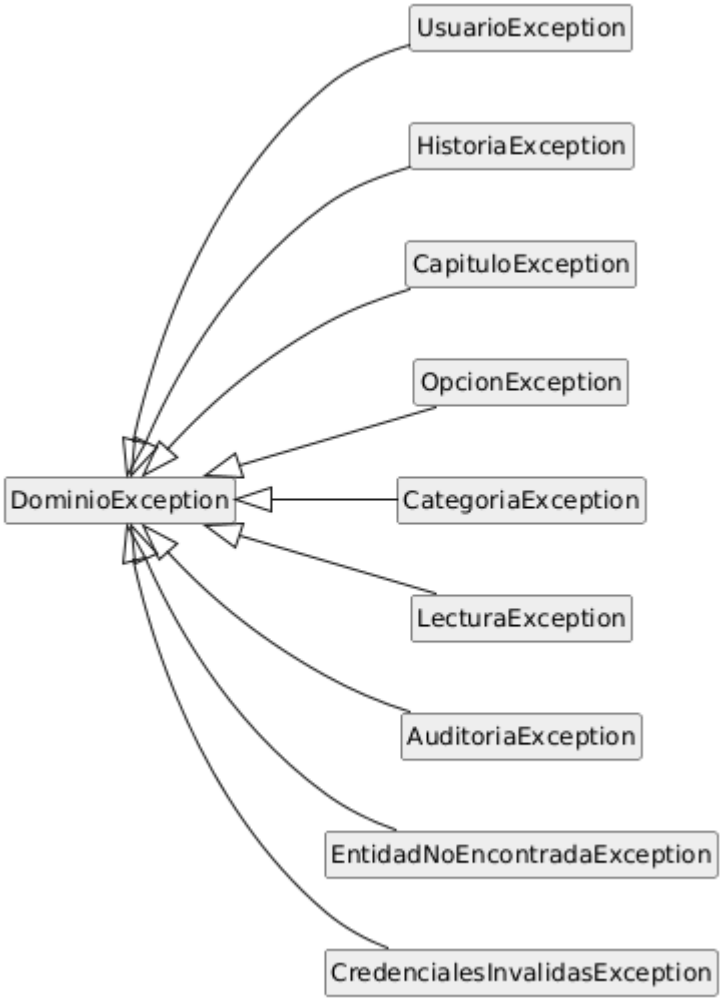
5.1. Dominio: entidades



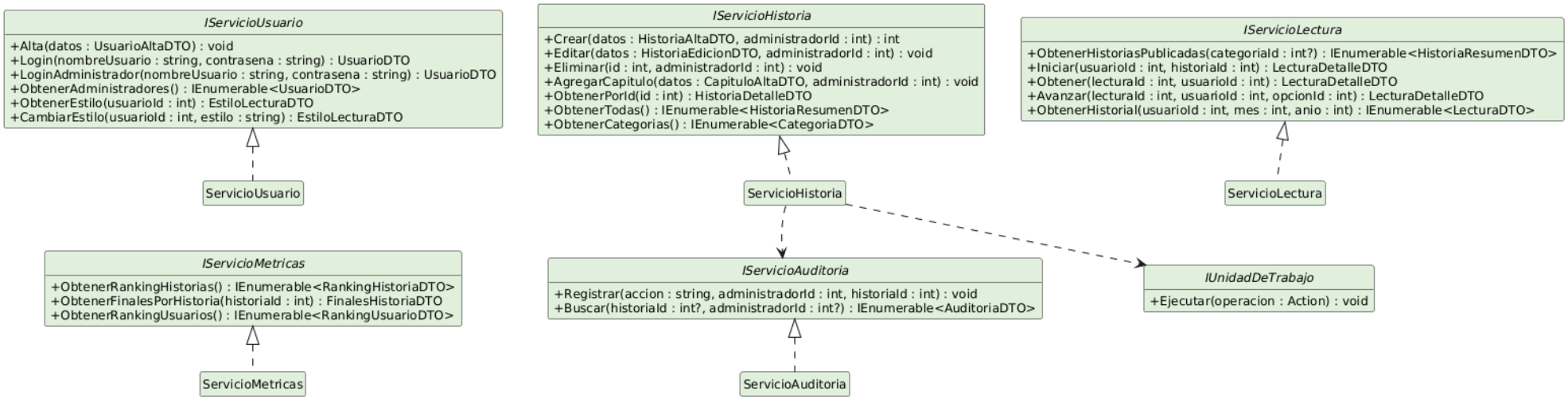
5.2. Dominio: repositorios y consultas



5.3. Dominio: excepciones



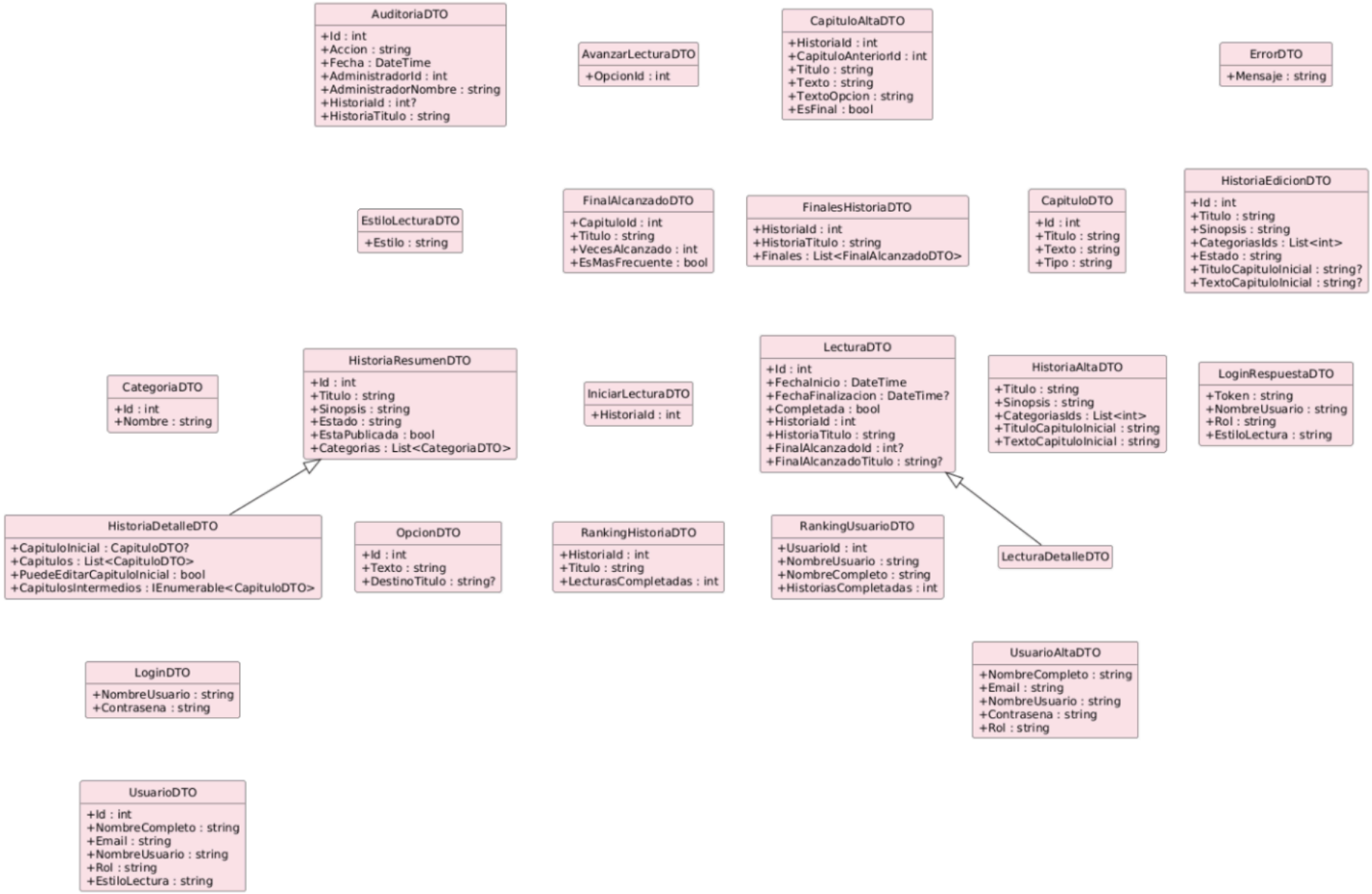
5.4. Aplicación: servicios



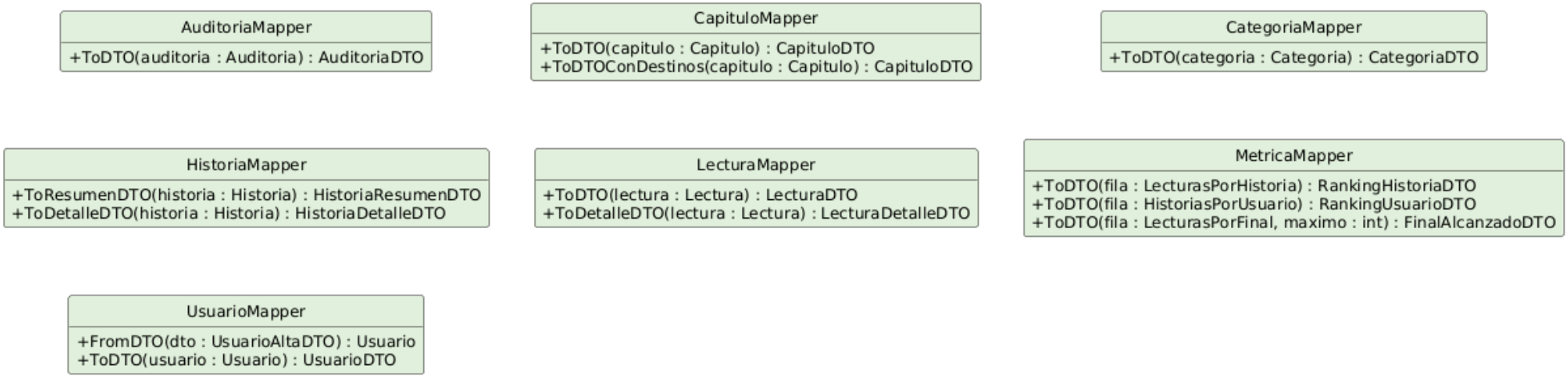
Dependencias de los servicios con los repositorios



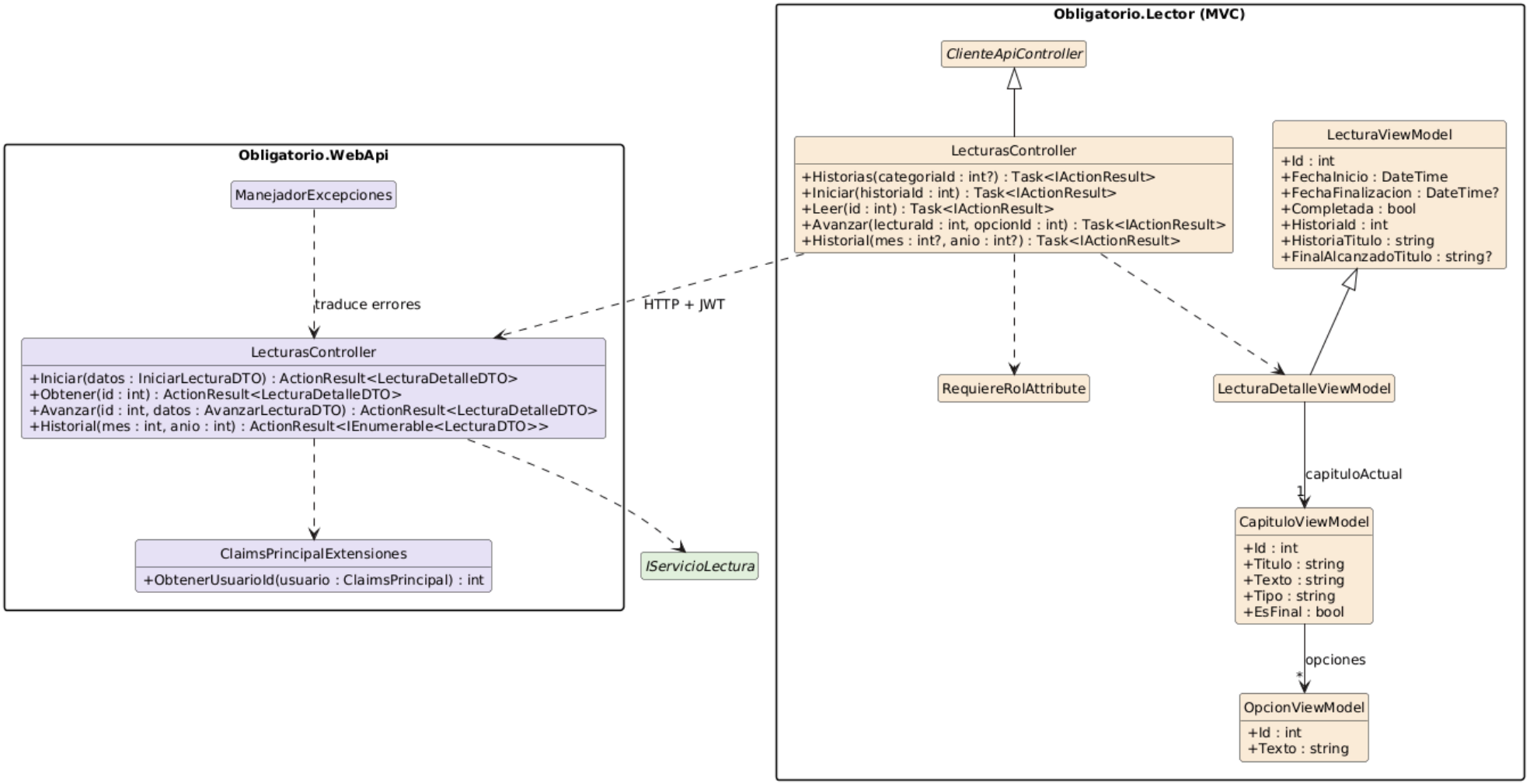
5.5. Aplicación: DTOs



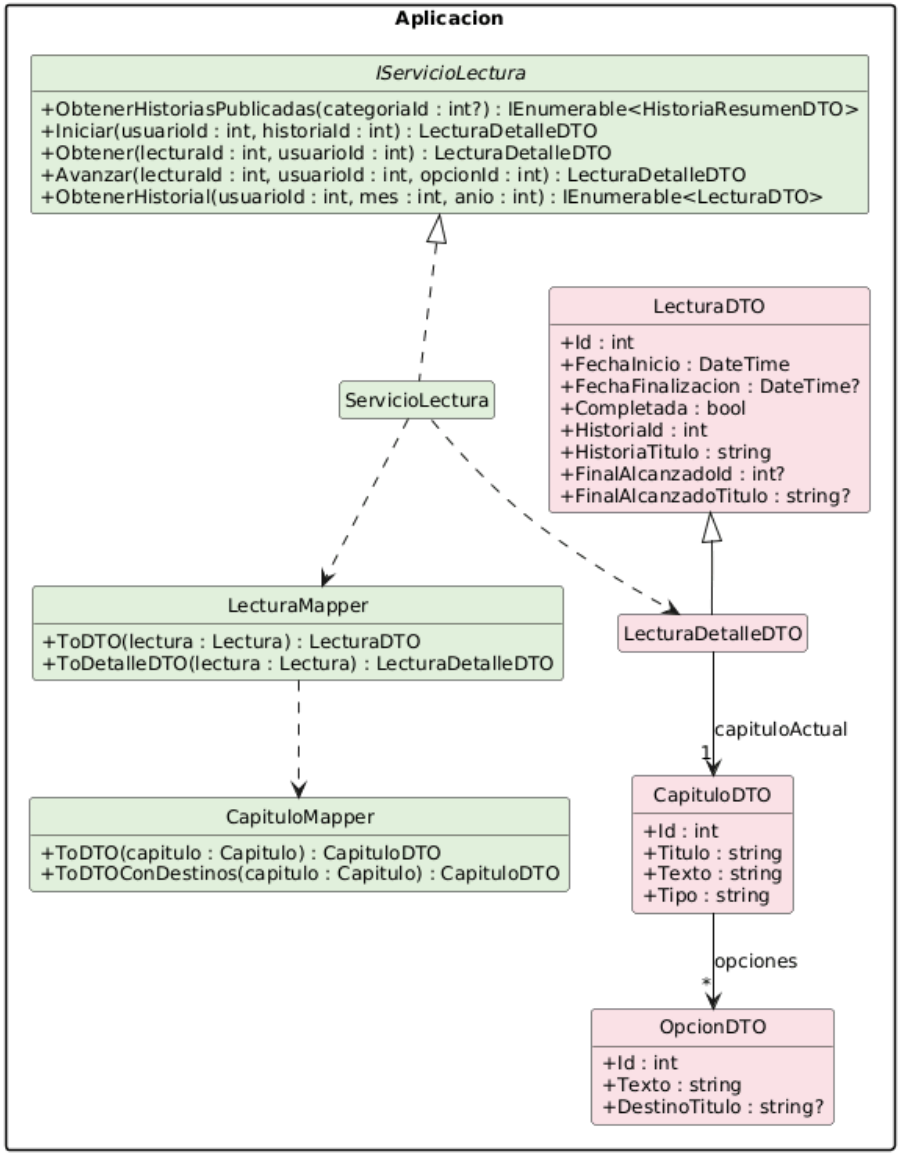
5.6. Aplicación: mappers



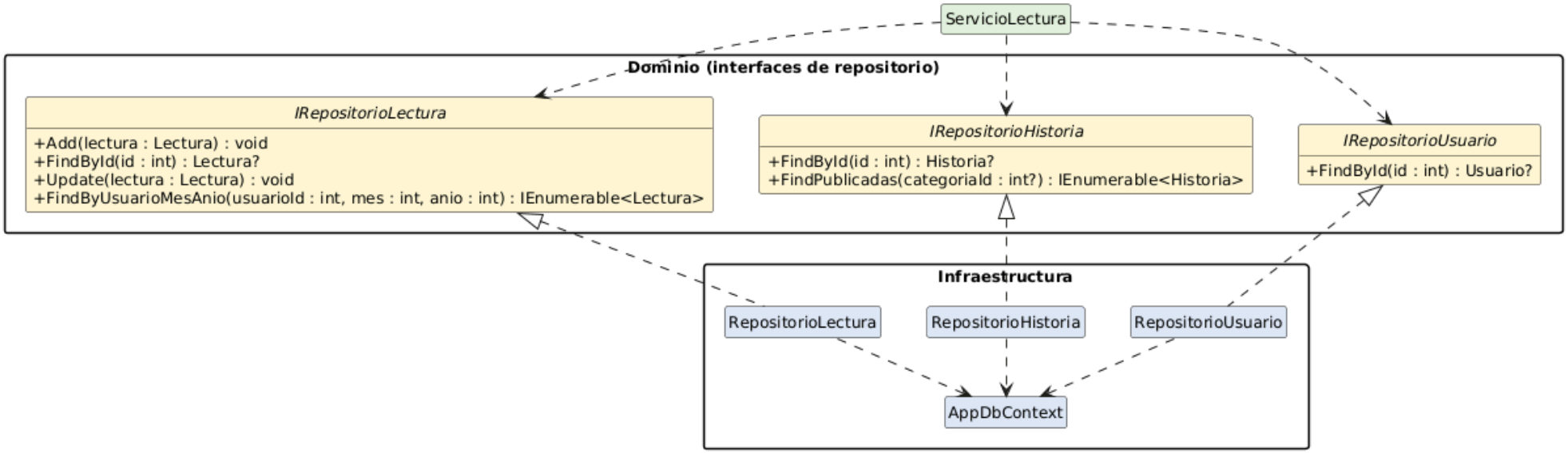
5.7. Lectura de historia: MVC y Web API



5.8. Lectura de historia: aplicación



5.9. Lectura de historia: repositorios e infraestructura

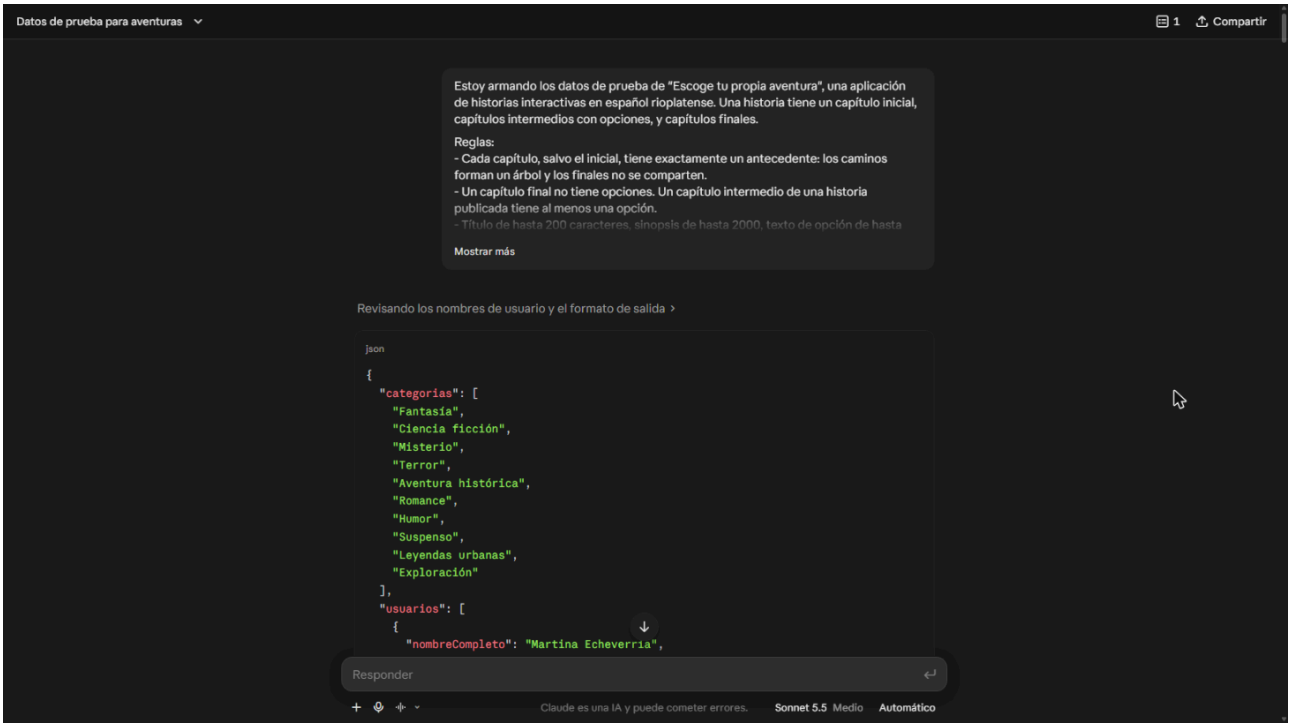


6. Generación de datos de prueba con IA generativa

Las historias, capítulos, opciones y finales de la precarga, junto con los usuarios y las lecturas de prueba, se generaron con ayuda de IA generativa.

Herramienta y modelo	Claude Sonnet 5.5 (claude.ai)
Secuencia de prompts	https://claude.ai/chat/87aba87b-fb80-4dd8-bd84-94a97438f96d

6.1. Captura de la conversación



6.2. Prompts utilizados

Prompt 1: contexto y datos base

```
Estoy armando los datos de prueba de "Escoge tu propia aventura", una aplicación de historias
interactivas en español rioplatense. Una historia tiene un capítulo inicial, capítulos intermedios
con opciones, y capítulos finales.

Reglas:
- Cada capítulo, salvo el inicial, tiene exactamente un antecedente: los caminos forman un árbol y
los finales no se comparten.
- Un capítulo final no tiene opciones. Un capítulo intermedio de una historia publicada tiene al
menos una opción.
- Título de hasta 200 caracteres, sinopsis de hasta 2000, texto de opción de hasta 300.
- Las contraseñas tienen al menos 8 caracteres, con mayúscula, minúscula, número y carácter
especial.
- Todo el contenido tiene que ser original, sin personajes ni obras existentes.

Generá solo este JSON, sin historias todavía:
{"categorias": [10 nombres], "usuarios": [{"nombreCompleto", "email", "nombreUsuario",
"contrasena", "rol", "estilo"}]}
```

```
Quiero 12 usuarios con emails @aventura.example: admin1 y admin2 con rol ADMINISTRADOR y contraseña
Admin123!, y 10 con rol LECTOR y contraseña Lector123!, entre ellos lector1 y nuevo. El campo
estilo es CLARO u OSCURO, con algunos de cada uno.
```

Prompt 2: historias

```
Agregá al JSON la lista "historias" y dos listas más: "ediciones": [{"historia", "admin", "fecha"}]
con 3 elementos, y "bajas": [{"titulo", "admin", "creada", "baja"}] con 1 elemento. Formato de cada
historia:

{"clave", "titulo", "estado", "admin", "creada", "categorias": [...], "sinopsis",
 "capitulos": [{"clave", "titulo", "texto", "opciones": [{"texto", "destino"}]}, {"clave",
"titulo", "texto", "final": true}]}

El primer capítulo de la lista es el inicial. "destino" es la clave de otro capítulo de la misma
historia. "admin" es admin1 o admin2. Las fechas tienen formato 2026-07-06T10:00:00, entre julio y
principios de noviembre de 2026.

Quiero 13 historias, todas distintas en tema y usando las 10 categorías:
- 11 PUBLICADAS, con 2 a 4 finales cada una. Una de ellas, con 4 finales.
- 1 BORRADOR completa, lista para publicar.
- 1 BORRADOR que tenga solo el capítulo inicial, con "opciones": [].
Los textos de 2 o 3 oraciones, coherentes, y los finales titulados "Final: ...". Devolvé el JSON
completo.
```

Prompt 3: datos de uso

```
Ahora generá uso.json con este formato: {"lecturas": [["usuario", "clave de historia", "clave del
capítulo donde quedó", "fecha de inicio", minutos]]}. Los minutos son el tiempo hasta llegar al
final; si la lectura quedó en un capítulo intermedio, van en null.

Quiero entre 30 y 40 lecturas, de agosto a principios de noviembre de 2026, posteriores a la
creación de cada historia, que permitan probar las métricas:
- Ranking de historias por lecturas completadas, con cantidades distintas y algún empate.
- La historia de 4 finales con 9 lecturas completadas, repartidas 5, 3 y 1, y un final nunca
alcanzado.
- Una historia publicada sin ninguna lectura y otra solo con lecturas sin terminar: ninguna de las
dos debe aparecer en el ranking.
- Una historia con dos finales empatados.
- Ranking de usuarios por historias distintas completadas, con algún usuario que repita una misma
historia. El usuario "nuevo" no lee nada.
- lector1 con lecturas en varios meses y ninguna en julio, para probar el historial.
Solo los LECTOR leen, y solo historias PUBLICADAS.
```

6.3. Del resultado de la IA al script

La IA devuelve dos archivos JSON. `BaseDeDatos/generador/generar_script.py` los valida y arma `BaseDeDatos/Obligatorio.sql`, que contiene el esquema y los inserts. En ese script, los comentarios separan la precarga generada con IA de los datos que surgen del uso. Los Id los genera la base de datos: cada Id nuevo se guarda con `SCOPE_IDENTITY()` y se reutiliza en las claves foráneas.

6.4. Controles

- Antes de generar el script se validan largos, emails, contraseñas, roles, que los caminos de cada historia formen un árbol, las reglas de publicación, que solo los lectores lean historias publicadas, y

el mínimo de 10 filas por tabla.

- Sobre la base creada se comprobó que todos los usuarios inician sesión, que las métricas coinciden con las calculadas desde los archivos y que se puede llegar a todos los finales de las historias publicadas.

6.5. Filas por tabla

Tabla	Filas
Categorias	10
Usuarios	12
Historias	13
CategoriaHistoria	17
Capitulos	63
Opciones	50
Auditorias	79
Lecturas	36

6.6. Datos para probar las métricas

RF09: historias más leídas

Historia	Lecturas completadas
El faro que no se apaga	9
Estación Orión-9	5
El taxi bajo el puente de la rambla	4
La casa de la calle Sauce	4
Carta en el tren de las siete	3
Expedición al Cerro del Eco	2
El gato contador de Barrio Sur	1
El pliego de la cuchilla	1
La última llamada de la noche	1

RF10: finales alcanzados por historia

Historia	Veces que se alcanzó cada final
El faro que no se apaga	5, 3, 1, 0
Estación Orión-9	3, 1, 1
La casa de la calle Sauce	2, 1, 1
El pliego de la cuchilla	1, 0
Carta en el tren de las siete	1, 1, 1

Historia	Veces que se alcanzó cada final
El gato contador de Barrio Sur	1, 0
La última llamada de la noche	1, 0, 0
El taxi bajo el puente de la rambla	2, 2, 0
Expedición al Cerro del Eco	1, 1, 0
El jardín de los relojes	0, 0
Don Robot se jubila	0, 0, 0

RF11: usuarios con más historias leídas

Usuario	Historias distintas completadas
lector1	7
lector2	4
lector3	4
lector4	4
lector5	3
lector6	3
lector7	3
lector8	1

7. Investigación para la selección de estilos (RF12)

7.1. Herramienta y método

- **IA utilizada:** Claude (modelo Opus 5.5, de Anthropic), en Claude Code.
- **Qué se le pidió:** comparar formas de ofrecer dos estilos visuales en una aplicación ASP.NET Core MVC con Bootstrap, elegir una y justificarla.
- **Cómo se verificó:** las afirmaciones técnicas se contrastaron con la documentación oficial (ver Fuentes), con el CSS de Bootstrap incluido en el proyecto y con mediciones de contraste sobre la aplicación en ejecución.

7.2. Qué había que resolver

1. Cómo cambiar los colores de toda la aplicación.

2. Dónde guardar la elección para que sea de cada usuario.

3. Cómo asegurar que los dos estilos se lean bien.

7.3. Cómo cambiar los colores

Alternativa	Ventajas	Desventajas	¿Se eligió?
Modos de color de Bootstrap 5.3 (`data-bs-theme`)	El proyecto ya usa esa versión. Un solo atributo cambia todos los componentes.	Hay que quitar las clases de color fijo y ajustar algunos contrastes.	Sí
`prefers-color-scheme`	No requiere guardar nada.	Sigue la configuración del sistema o del navegador, no una elección hecha en la aplicación. No cumple RF12 por sí sola.	No
Dos hojas de estilo	Separación total entre estilos.	Duplica CSS y hay que mantener ambas al día.	No
Variables CSS propias	Control total.	Obliga a redefinir lo que Bootstrap ya resuelve.	No

Bootstrap define sus colores como variables CSS y las redefine bajo `[data-bs-theme=dark]`. El atributo también se puede aplicar a un solo elemento: así se arma la vista previa de cada estilo en la pantalla de selección.

7.4. Dónde guardar la elección

Alternativa	Comportamiento	¿Se eligió?
`localStorage` (lo que propone el ejemplo de la documentación de Bootstrap)	Queda en el navegador, no en el usuario: dos personas en la misma computadora comparten el estilo, y se pierde al cambiar de dispositivo.	No

Alternativa	Comportamiento	¿Se eligió?
Base de datos, a través de la Web API	Queda asociada al usuario y lo acompaña en cualquier navegador.	Sí

El MVC guarda una copia del estilo en la sesión, para no llamar a la API en cada página. Como el atributo sale escrito desde el servidor en el HTML, la página no parpadea en el otro estilo mientras carga.

7.5. Contraste

WCAG 2.2, criterio 1.4.3 (nivel AA), exige un contraste de al menos 4.5:1 para texto normal, y 3:1 para texto grande. Activar el modo oscuro de Bootstrap no alcanzó: los botones de contorno quedaron por debajo del mínimo. Se corrigieron en `site.css` con los tonos claros que el propio Bootstrap define para ese modo.

Mediciones sobre las pantallas de lectura en ejecución:

Elemento	Claro	Oscuro antes del ajuste	Oscuro final
Texto del capítulo	15.43	11.85	11.85
Texto secundario (`text-muted`)	6.78	7.29	7.29
Enlaces del menú	6.98	7.32	7.32
Alerta de éxito	10.35	7.67	7.67
Botón principal	4.50	4.50	4.50
Botón de contorno primario	4.50	**3.43**	6.39
Botón de contorno secundario	4.69	**3.29**	6.74
Botón de contorno de peligro	4.53	**3.41**	6.10

En negrita, los valores que no cumplían. Los 4.50 del botón principal y del contorno primario superan apenas el mínimo (4.5008).

7.6. Implementación

Capa	Elemento	Responsabilidad
Dominio	`ValueObjects/EstiloLectura`	Value Object: solo admite `CLARO` u `OSCURO`, y es inmutable.
Dominio	`Usuario.CambiarEstilo`	Cambia el estilo del usuario. Todo usuario arranca en `CLARO`.
Infraestructura	`AppDbContext` (`OwnsOne`)	Guarda el Value Object como la columna `EstiloLectura` de `Usuarios`, con valor por defecto `CLARO`.
Aplicación	`ServicioUsuario`	`ObtenerEstilo` y `CambiarEstilo`.
Web API	`UsuariosController`	`GET` y `PUT /api/Usuarios/estilo`, solo para lectores. El usuario sale del token.

Capa	Elemento	Responsabilidad
Web API	`AuthController`	El login devuelve el estilo guardado.
MVC	`PreferenciasController`, `Preferencias/Estilo`	Pantalla de selección.
MVC	`_Layout.cshtml`	Escribe `data-bs-theme` en `<html>` según la sesión.

Para `EstiloLectura` se usó un tipo owned, igual que el Value Object visto en clase. La documentación actual de EF Core sugiere considerar los complex types para Value Objects. Con owned, una misma instancia no se puede compartir entre usuarios, por eso cada usuario tiene la suya.

7.7. Decisiones y limitaciones

- El estilo es para el rol Usuario, como pide RF12. Los administradores ven siempre el estilo claro en el MVC de lectura.
- Antes de iniciar sesión se ve el estilo claro, porque todavía no se sabe quién es el usuario.
- Si el usuario cambia el estilo desde otro navegador, la sesión abierta lo toma al volver a iniciar sesión o al entrar a la pantalla de estilo.
- El proyecto usa Bootstrap 5.3.3. La documentación de Bootstrap 6.0 cambia el comportamiento por defecto y sigue la preferencia del sistema, así que una actualización de versión requeriría revisar esto.

7.8. Fuentes

- Color modes - Bootstrap 5.3
- Color modes - Bootstrap 6.0
- Understanding SC 1.4.3: Contrast (Minimum) - W3C, WCAG 2.2
- prefers-color-scheme - MDN
- Owned Entity Types - EF Core
- Complex Types - EF Core